Justin Vince B. Alcayde
Role: Backend Developer (API Development)

Specific Tasks Performed

The majority of my work was building the REST API *(api.php)*, setting up the MySQL database, and giving the api.php commands to do anything from looking up public artist data, to giving admins the ability to approve or delete artists. I also developed the code to connect to the database and used MySQL to return all responses in JSON format.

Code Contributions

I used a switch-case statement to record any GET, POST, PUT and DELETE requests made to the server in one file, and a syncJSON() function that I called each time an admin modifies the database to ensure that the verified_artists.json file is updated with the data in the SQL database. I sanitize inputs before putting it into the database by using prepared statements ($stmt->prepare) to prevent SQL injection.

Challenges Encountered & Problem Solving

The hardest part of the task was routing the different types of HTTP requests (GET, POST, PUT, DELETE) to the right part of the code. My if-else blocks were getting really messy and hard to debug. I ended up refactoring it into a switch ($method) block, which made it much easier to read. I spent a lot of time hooking up the X-Admin-Token header correctly. I was getting 401 Unauthorized errors until I realized I wasn't grabbing that header correctly in the PHP.

I also had a hard time on the "Approval" part. I had difficulty rolling an user from the pending table to the verified table without losing any data in the tables. I later figured out I could work around this by using an INSERT INTO ... SELECT query and deleting the record from the pending table. The API wasn't accepting the admin token either. After some debugging, I realized I should be using $_SERVER['HTTP_X_ADMIN_TOKEN'] to retrieve the value of the custom header.

Jesslyn E. Dayto
Role: Frontend/Client Developer (Web Interface/Application)

Specific Tasks Performed

Developing the web client *(index.html)* was the task assigned to me. It mostly involves structuring HTML and CSS, as well as creating the JavaScript that connected our PHP backend to the website's interface. I worked on the index.html file, where I created the search bar and the registration form (for the public), as well as the admin dashboard. The whole idea behind the web interface was that the admin could approve, edit, or delete artists and do their job without ever needing to look at the database.

Code Contributions

The first thing I did was handle the asynchronous requests using JavaScript's fetch() API with async/await. I created a loadAdminData() method that makes two fetch calls at the same time—one to get the list of approved artists and another to get the pending requests. I also built a simple login system that saves the admin token to localStorage after a successful login. Finally, I made sure every admin request included the X-Admin-Token in the header so it would work properly with our Backend Developer's API.

Challenges Encountered & Problem Solving

Getting the fetch() calls to work properly was a challenge, and there was some time (at the initial testing) when the UI wouldn't refresh correctly when an artist was deleted from the database. Eventually, I figured out that it was my code for the asynchronous calls that were causing the issue, so I converted everything to use async/await, and started using localStorage to keep the admin logged in. A further issue was that the JSON body of the "Edit Bio" request would not be read by the PHP backend unless the Content-Type: application/json header was sent to prevent it from being read from the php://input stream. Nonetheless, the less crucial but overtly annoying problem is when xampp acted out of nowhere and decided to crash. Thus, I had difficulty with testing the web app’s connectivity with the API. My resolution to this was to delete all files related to xampp and install it all again to get a clean slate.

Kirsten Jade C. Armero
Role: Client Developer (Python Application)

Specific Tasks Performed

I built the Python-based client application *(admin_tool.py)* that interacts with the API to show that all the functionality in our web application was actually platform-independent. This client does almost everything our web application does. You can also check out artists using the menu system, or log in as an administrator to manage the pending artists list.

Code Contributions

Similar to the member assigned to do Client 1, I was mainly responsible for developing the python terminal or CLI based application. This is the admin_tool.py file in the school-app source folder, and like its web version (Client 1), it must interact with the API to be fully functional. To be specific, the script supports public functions like artist verification, registration requests, plus an admin dashboard with numbered menus for approving users, editing bios, and deleting artists.

Moreover, I used the requests library to ease HTTP requests to our API. A menu that provided options for public and admin tasks was created using a while True loop. Additionally, I implemented a feature that fetches the pending list as a numbered list, so that the admin can approve the user by their number rather than typing their username.

Challenges Encountered & Problem Solving

At first I didn't understand how to make HTTP requests from Python, such as the PUT and DELETE requests, how to format JSON payloads, and how to handle connection errors so that the script wouldn't crash if XAMPP was off. So I added some error handling using try-except statements and checked the response.status_code before attempting to display the results. This makes the Python client much more robust and easier to use.